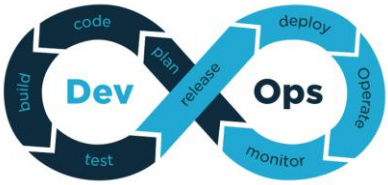
FATIMA JINNAH WOMEN UNIVERSITY
RAWALPINDI

Cloud DevOps



Lab 03

Working with Git History, Stashing, and Reverting Commits

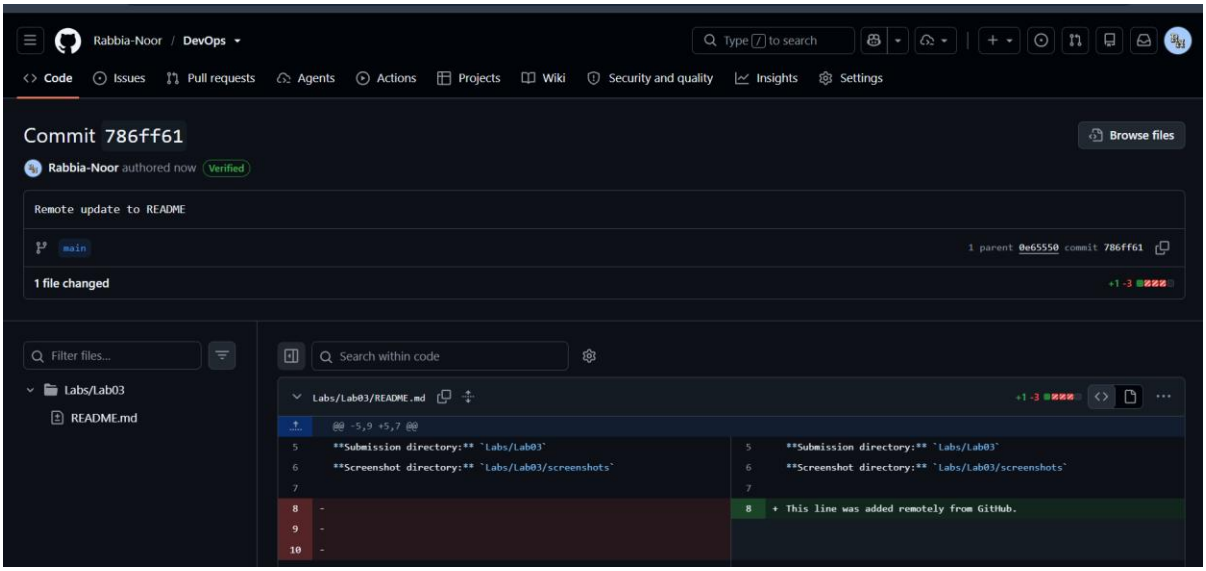
Submitted by: Rabbia Noor
Registration No.: 2023-BCS-070
Course Code: CS-649
GitHub Username: Rabbia-Noor
Section: A

Table of Contents

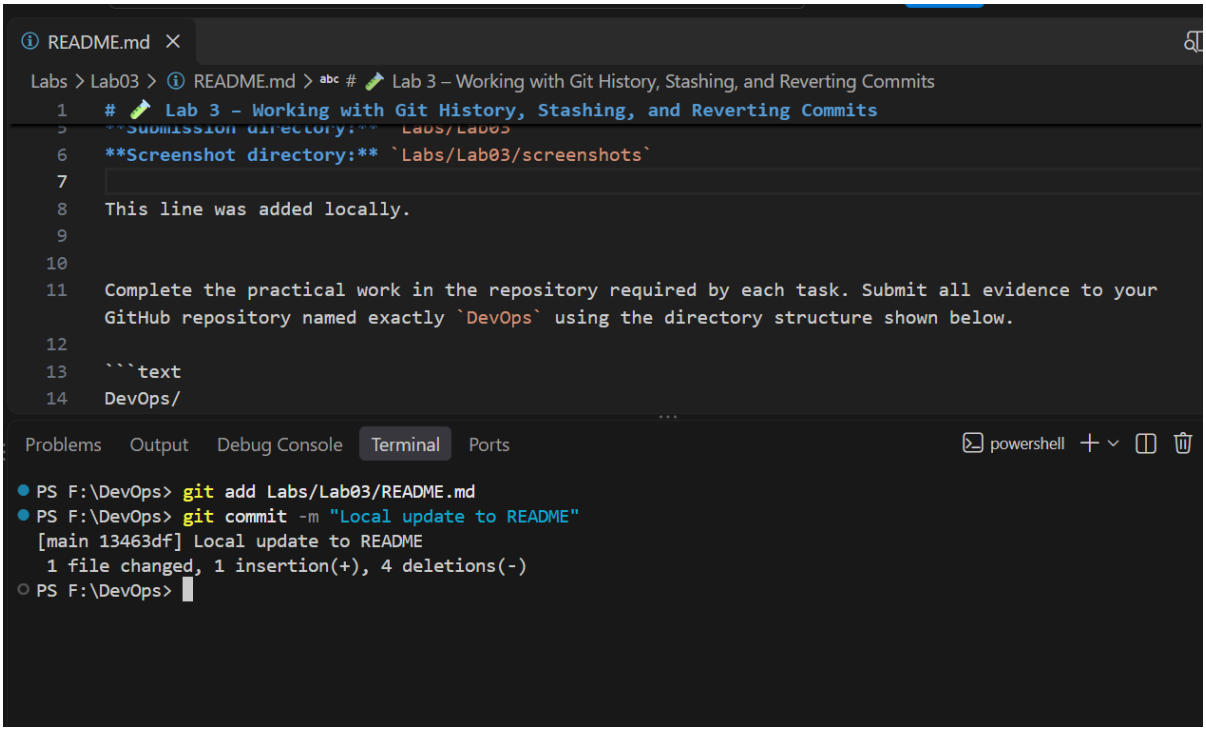
Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase) 3
Task 2 – Creating and Resolving Merge Conflicts Manually 5
Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files..... 8
Task 4 – Create Temporary Changes and Use git stash 11
Task 5 – Checkout a Specific Commit Using git log..... 12
Task 6 – Resetting Commits (Soft vs Hard Reset) 13
Task 7 – Amending the Last Commit..... 19
Task 8 – Reverting a Commit (Safe Undo on Remote Branch) 20
Task 9 – Force Push (With Caution) 21
Task 10 – Running Gitea in GitHub Codespaces via Docker Compose 22
Task 11 – Creating a GitHub Pages Portfolio Site 25

Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

Step 1: Edited README.md directly on GitHub and committed the change. This creates a new commit on the remote that does not exist in the local repository.



Step 2: Edited README.md on the local machine and committed it using git add and git commit. Now the local and remote branches each have a different new commit.



Step 3: Tried git push origin main, but it was rejected. Git refuses the push because the remote contains work that is not available locally.

```
 1 file changed, 1 insertion(+), 1 deletion(-)
PS F:\DevOps> git push origin main
To https://github.com/Rabbia-Noor/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/Rabbia-Noor/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS F:\DevOps>
```

Step 4: Ran git pull --no-rebase origin main to fetch the remote commit. Git merged both histories and created a merge commit.

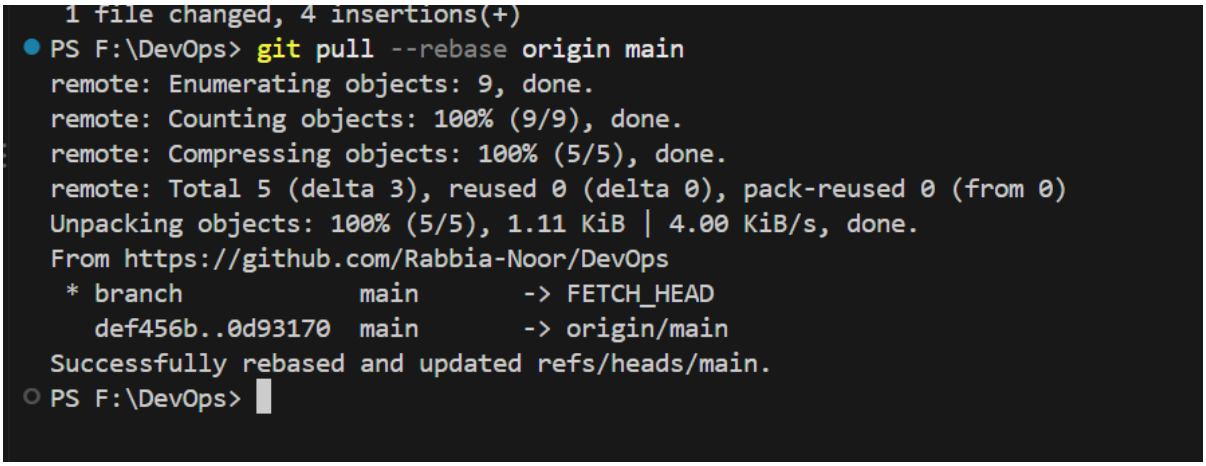
```
Problems  Output  Debug Console  Terminal  Ports                                    git

Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
~
~
~
~
~
~
.git/MERGE_MSG [unix] (19:55 16/10/2026)
"/f/DevOps/.git/MERGE_MSG" [unix] 6L, 287B
```

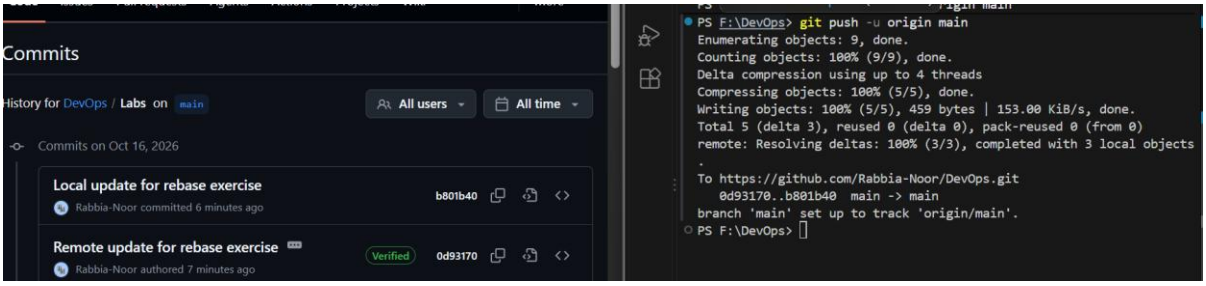
Step 5: Pushed again with git push -u origin main. The push succeeded because the local branch now contains the remote changes.

```
PS F:\DevOps> git push -u origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 4 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 915 bytes | 183.00 KiB/s, done.
Total 10 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 3 local objects.
To https://github.com/Rabbia-Noor/DevOps.git
   c0fe0c7..def456b  main -> main
branch 'main' set up to track 'origin/main'.
PS F:\DevOps>
```

Step 6: Made another remote and local edit, then ran git pull --rebase origin main. Git replayed the local commit on top of the remote commit, keeping the history linear.

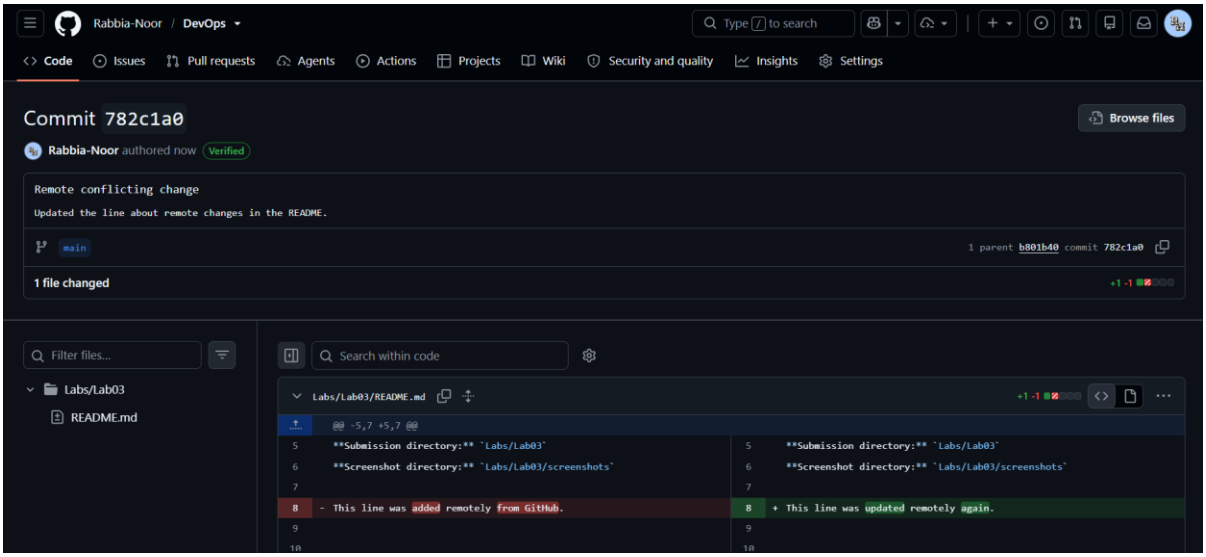


Step 7: Pushed the rebased commits to GitHub successfully with git push -u origin main.

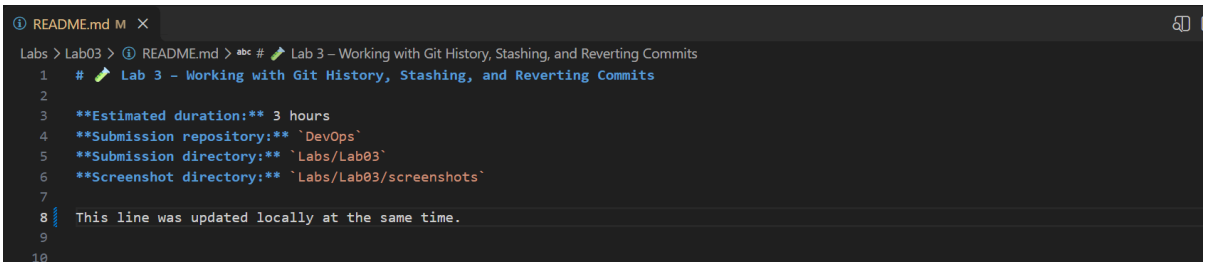


Task 2 – Creating and Resolving Merge Conflicts Manually

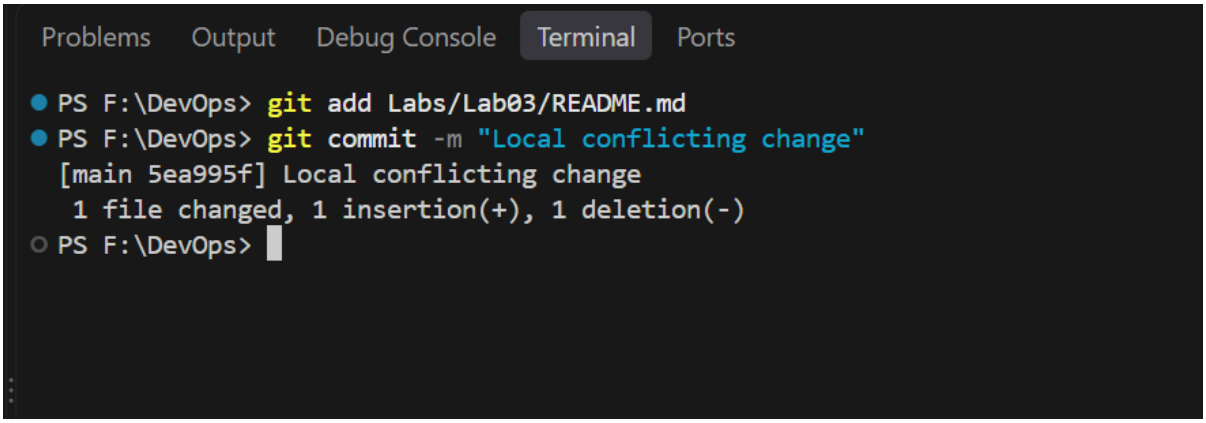
Step 1: Changed an existing line in README.md on GitHub and committed it as "Remote conflicting change".



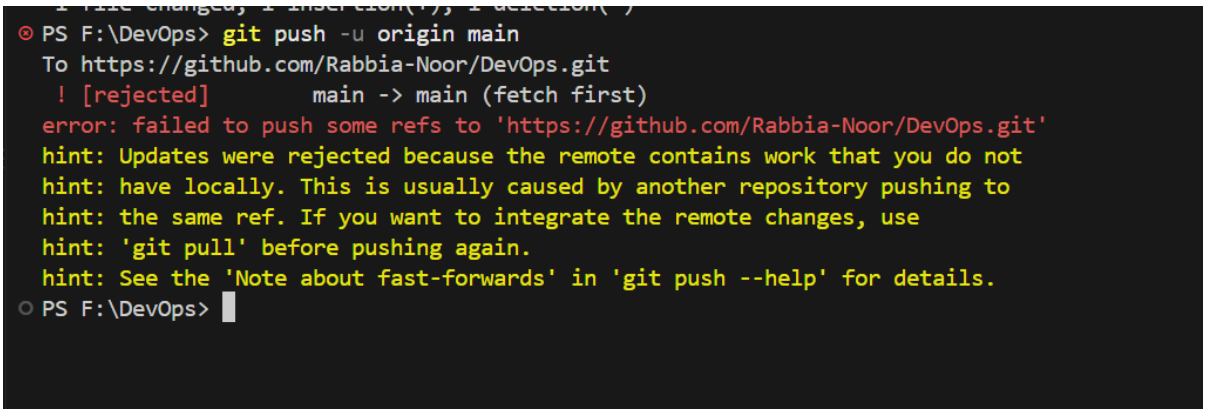
Step 2: Changed the same line in the local README.md to different text, which will conflict with the remote change.



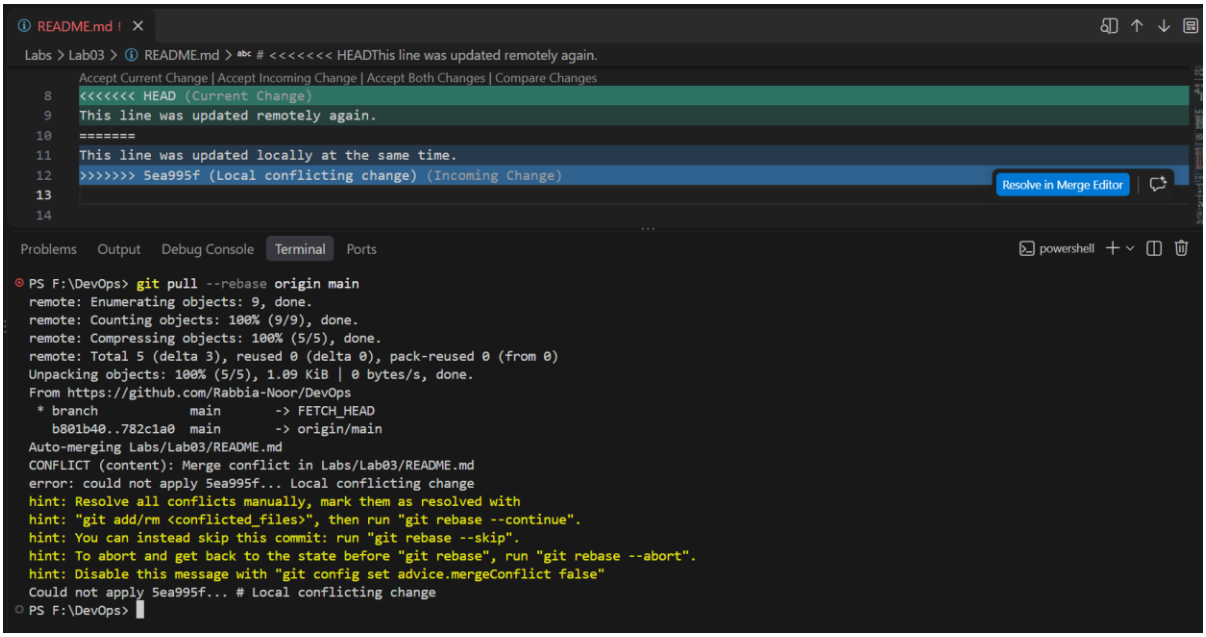
Step 3: Staged and committed the local change with the message "Local conflicting change".



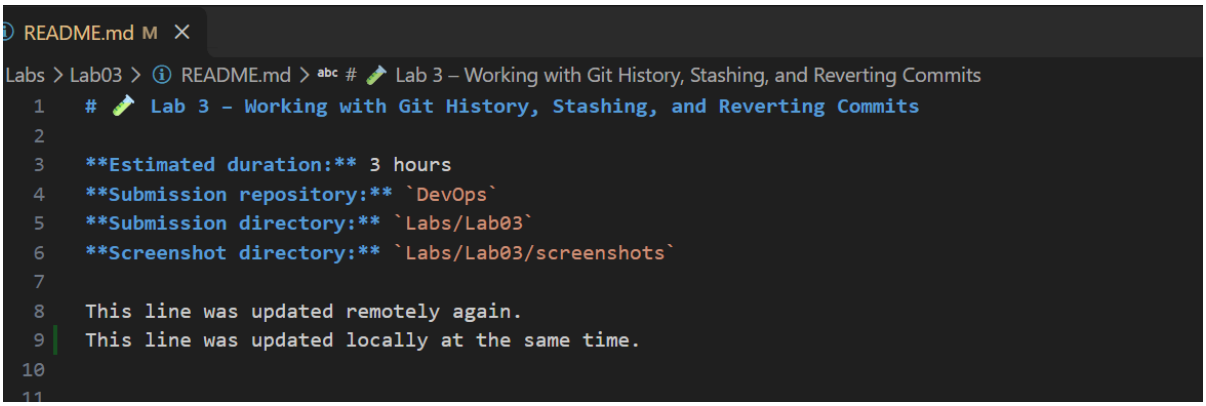
Step 4: Tried to push, but the remote rejected it because the same line was changed differently on GitHub.



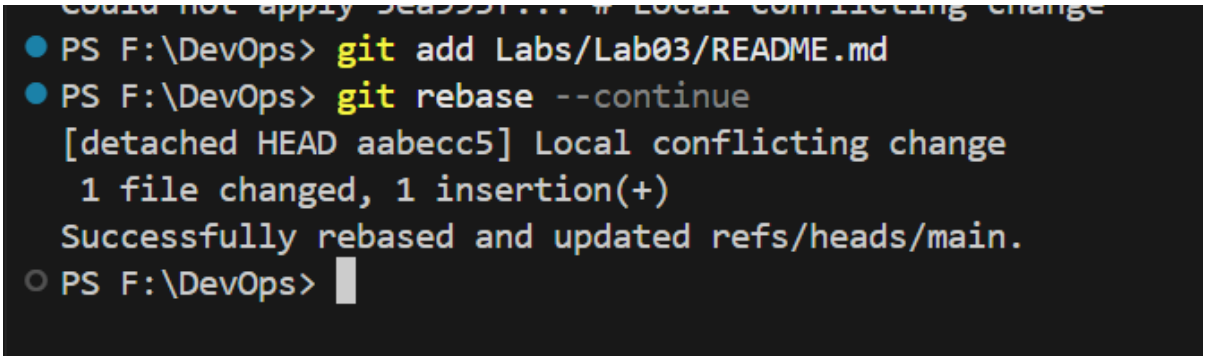
Step 5: Ran git pull --rebase origin main. Git paused the rebase and reported a conflict in README.md.



Step 6: Opened README.md, removed the conflict markers (<<<<<<<, =======, >>>>>>>) and kept the correct line.



Step 7: Marked the file as resolved with git add and ran git rebase --continue. The rebase finished successfully.



Step 8: Pushed the resolved commit to the remote with git push -u origin main.

```
PS F:\DevOps> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 456 bytes | 114.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Rabbia-Noor/DevOps.git
   782c1a0..aabecc5  main -> main
branch 'main' set up to track 'origin/main'.
PS F:\DevOps>
```

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

Step 1: Created a textfiles folder with a.txt, b.txt and c.txt, then added, committed and pushed it to GitHub.

```
PS F:\DevOps> mkdir textfiles


    Directory: F:\DevOps

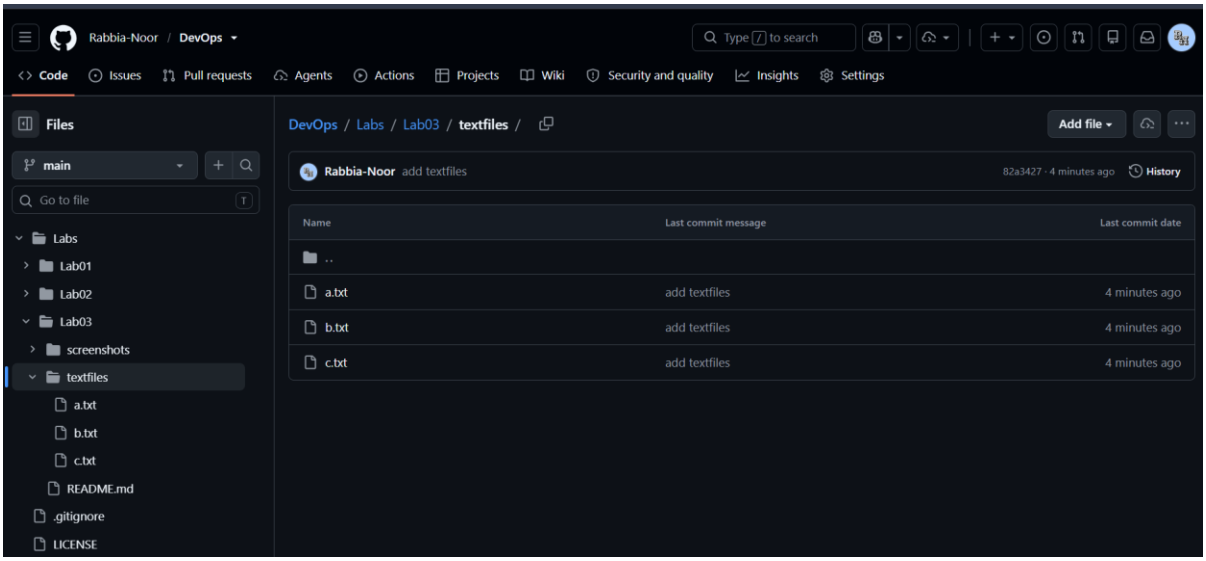

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/16/2026   8:59 PM                textfiles


PS F:\DevOps> echo "File A content" > textfiles/a.txt
>> echo "File B content" > textfiles/b.txt
>> echo "File C content" > textfiles/c.txt
PS F:\DevOps> git add .
PS F:\DevOps> git commit -m "Added textfiles directory with three files"
>>
[main 09c00cb] Added textfiles directory with three files
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 textfiles/a.txt
 create mode 100644 textfiles/b.txt
 create mode 100644 textfiles/c.txt
PS F:\DevOps> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 495 bytes | 123.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Rabbia-Noor/DevOps.git
   aabecc5..09c00cb  main -> main
PS F:\DevOps>
```

Step 2: Created a .gitignore file containing textfiles/*, then committed and pushed it.

```
PS F:\DevOps> echo "Labs/Lab03/textfiles/*" >> .gitignore
PS F:\DevOps> git add .gitignore
PS F:\DevOps> git commit -m "Added .gitignore to ignore textfiles directory"
[main 096e0e2] Added .gitignore to ignore textfiles directory
 1 file changed, 0 insertions(+), 0 deletions(-)
PS F:\DevOps> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 373 bytes | 124.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Rabbia-Noor/DevOps.git
   82a3427..096e0e2  main -> main
PS F:\DevOps>
```

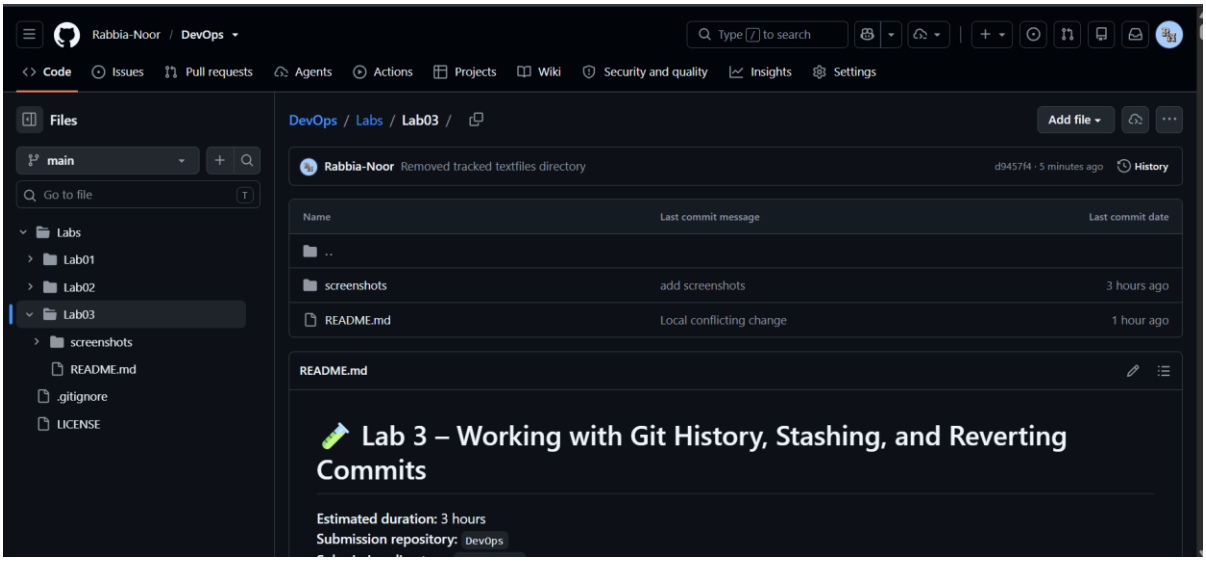
Step 3: The textfiles folder is still visible on GitHub, because .gitignore does not affect files that are already tracked.



Step 4: Ran git rm -r --cached textfiles to stop tracking the folder, then committed and pushed the change.

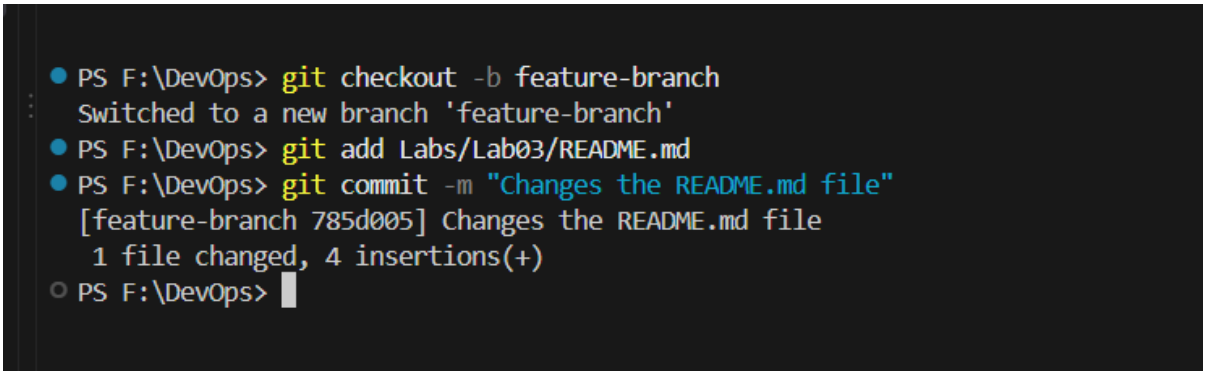
```
PS F:\DevOps> git rm -r --cached Labs/Lab03/textfiles
rm 'Labs/Lab03/textfiles/a.txt'
rm 'Labs/Lab03/textfiles/b.txt'
rm 'Labs/Lab03/textfiles/c.txt'
PS F:\DevOps> echo "/Labs/Lab03/textfiles/*" >> .gitignore
PS F:\DevOps> echo "/Labs/Lab03/textfiles/*" >> .gitignore
PS F:\DevOps> echo "/Labs/Lab03/textfiles/*" >> .gitignore
PS F:\DevOps> [System.IO.File]::WriteAllText("$PWD\.gitignore", "/Labs/Lab03/textfiles/*`n", [System.Text.UTF8Encoding]::new($false))
PS F:\DevOps> git check-ignore -v --no-index Labs/Lab03/textfiles/a.txt
.gitignore:1:/Labs/Lab03/textfiles/*    Labs/Lab03/textfiles/a.txt
PS F:\DevOps> git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
PS F:\DevOps> git commit -m "Removed tracked textfiles directory"
[main d9457f4] Removed tracked textfiles directory
 4 files changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 Labs/Lab03/textfiles/a.txt
 delete mode 100644 Labs/Lab03/textfiles/b.txt
 delete mode 100644 Labs/Lab03/textfiles/c.txt
PS F:\DevOps> git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 434 bytes | 144.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Rabbia-Noor/DevOps.git
   6e44ddd..d9457f4  main -> main
PS F:\DevOps>
```

Step 5: The textfiles folder is now removed from GitHub, while the files still exist on the local machine.

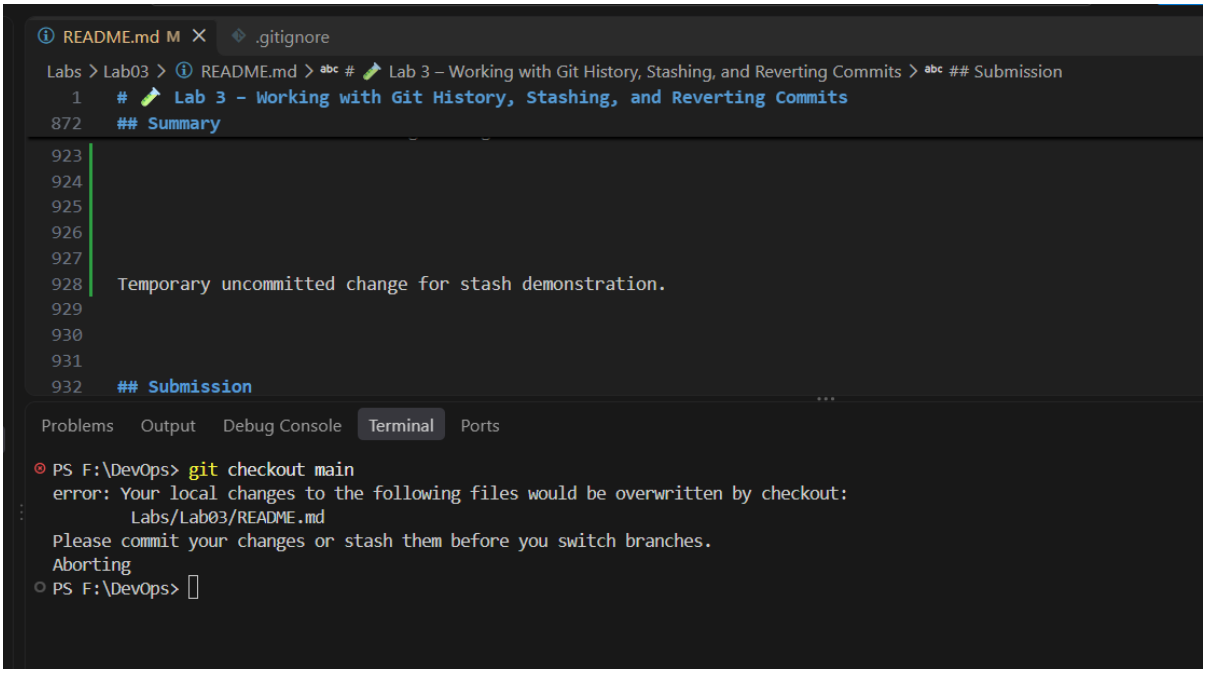


Task 4 – Create Temporary Changes and Use git stash

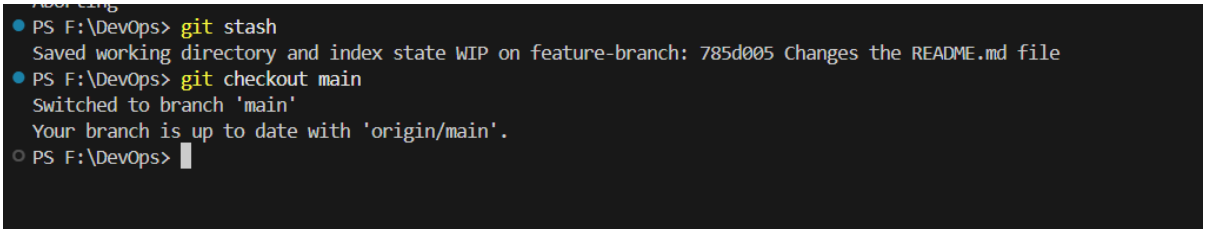
Step 1: Created feature-branch, added some lines to README.md and committed the change.



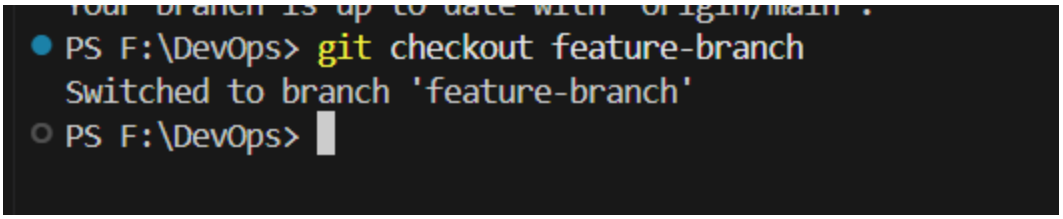
Step 2: Edited README.md again without committing and tried git checkout main. Git blocked the switch to avoid overwriting the local changes.



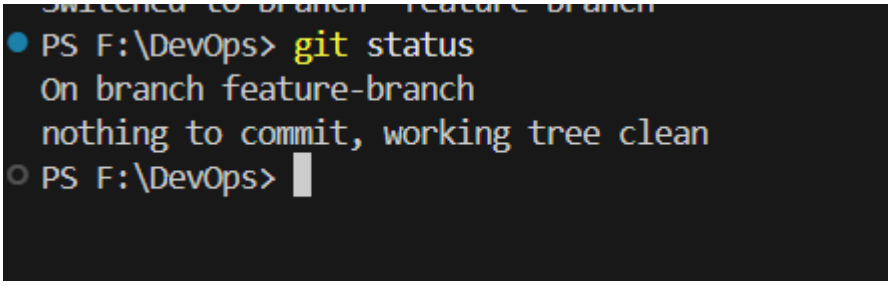
Step 3: Ran git stash to save the uncommitted changes, then git checkout main. The branch switch now works.



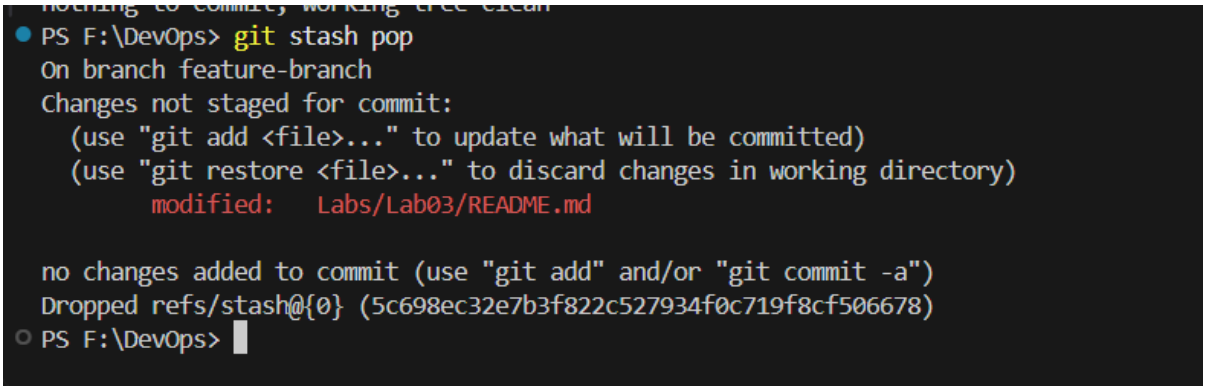
Step 4: Returned to feature-branch using git checkout feature-branch.



Step 5: git status reports a clean working tree, because the edits are safely saved in the stash.

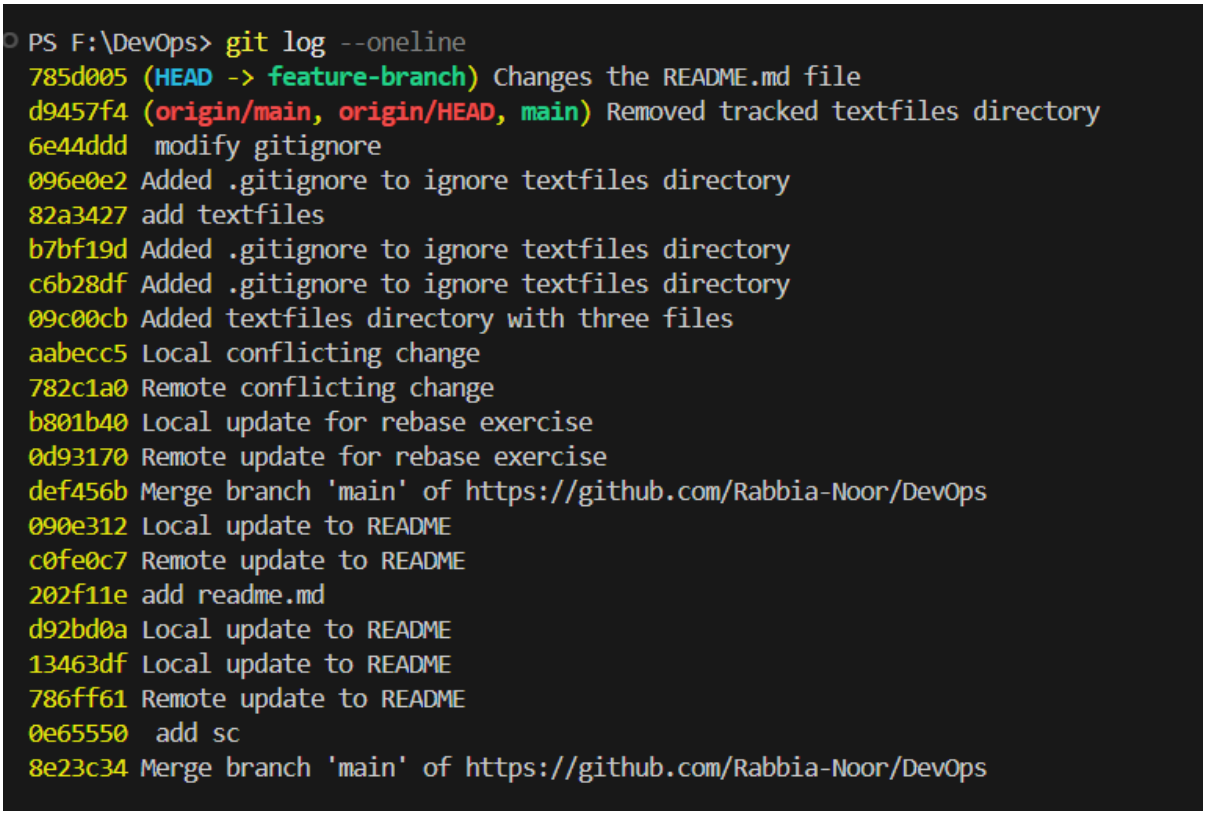


Step 6: Ran git stash pop to restore the stashed changes. The earlier edits are back in README.md.

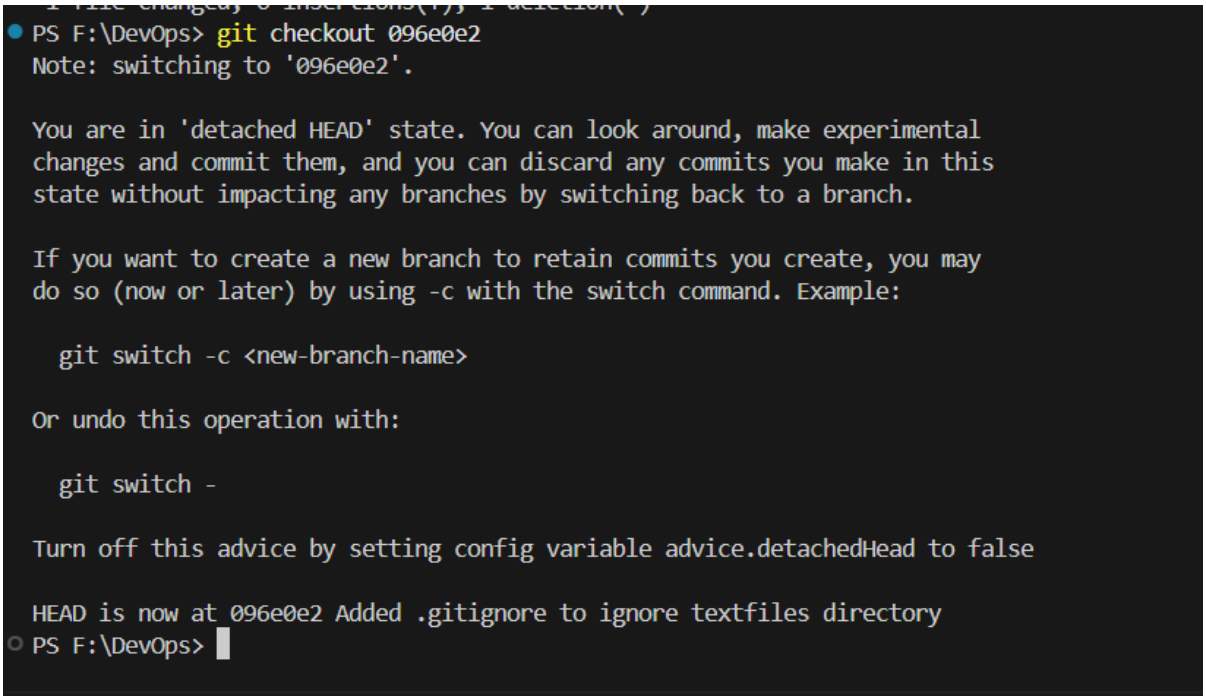


Task 5 – Checkout a Specific Commit Using git log

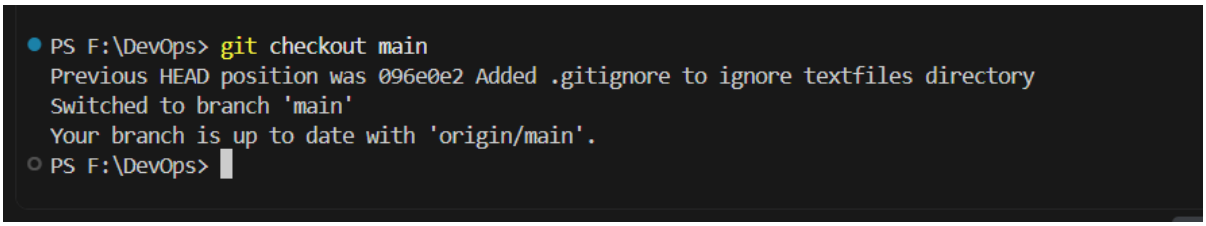
Step 1: Listed the commit history with git log --oneline to find a previous commit hash.



Step 2: Checked out an older commit using its hash. Git shows the detached HEAD state, which means HEAD is not on any branch.

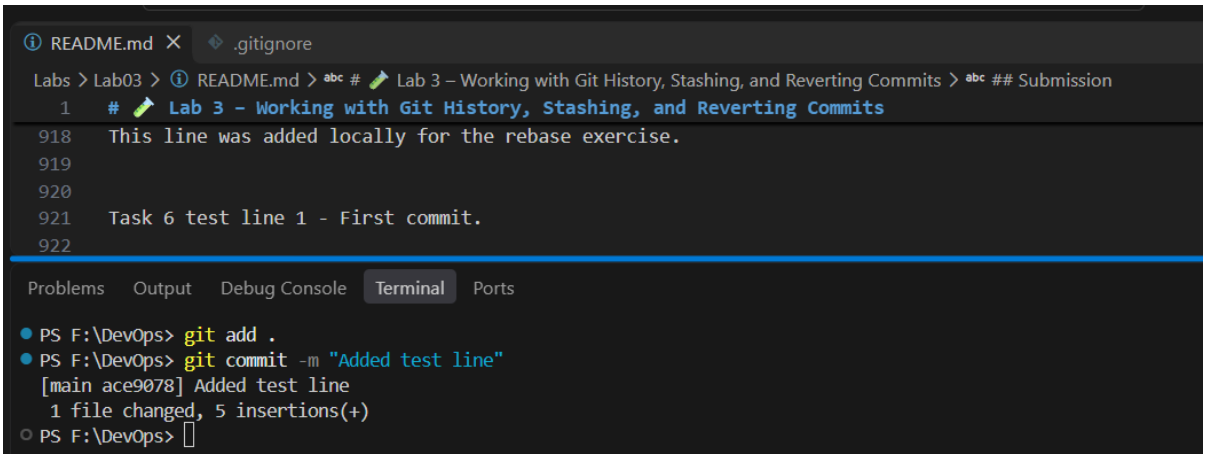


Step 3: Returned to the main branch with git checkout main.

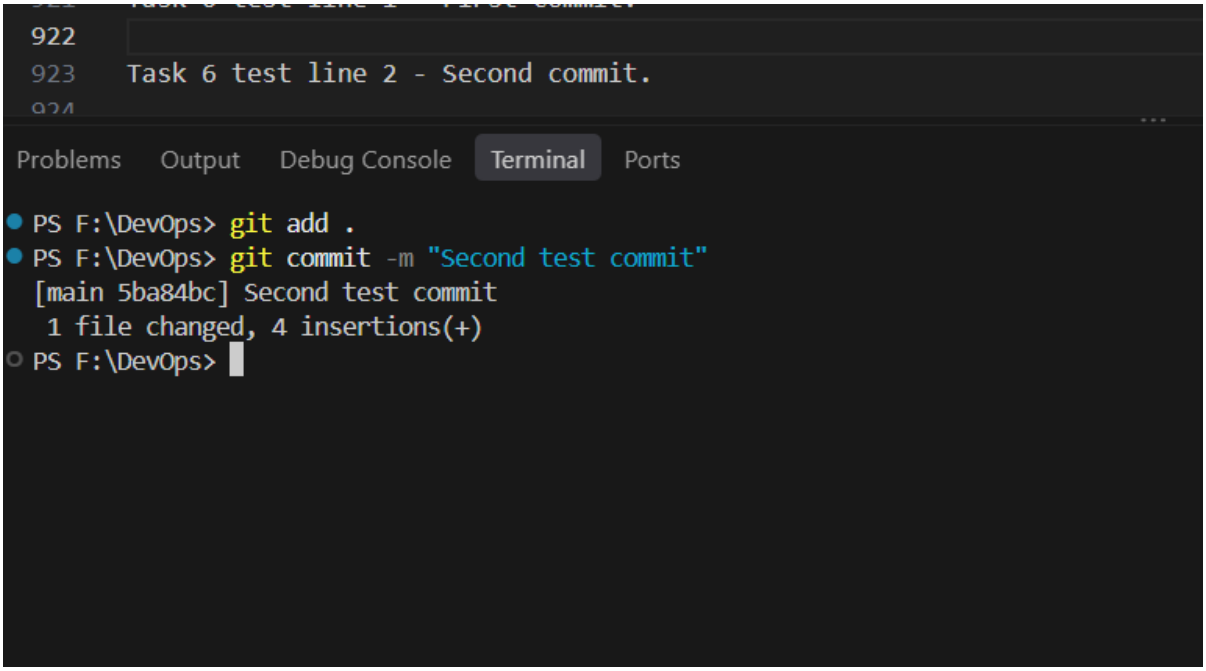


Task 6 – Resetting Commits (Soft vs Hard Reset)

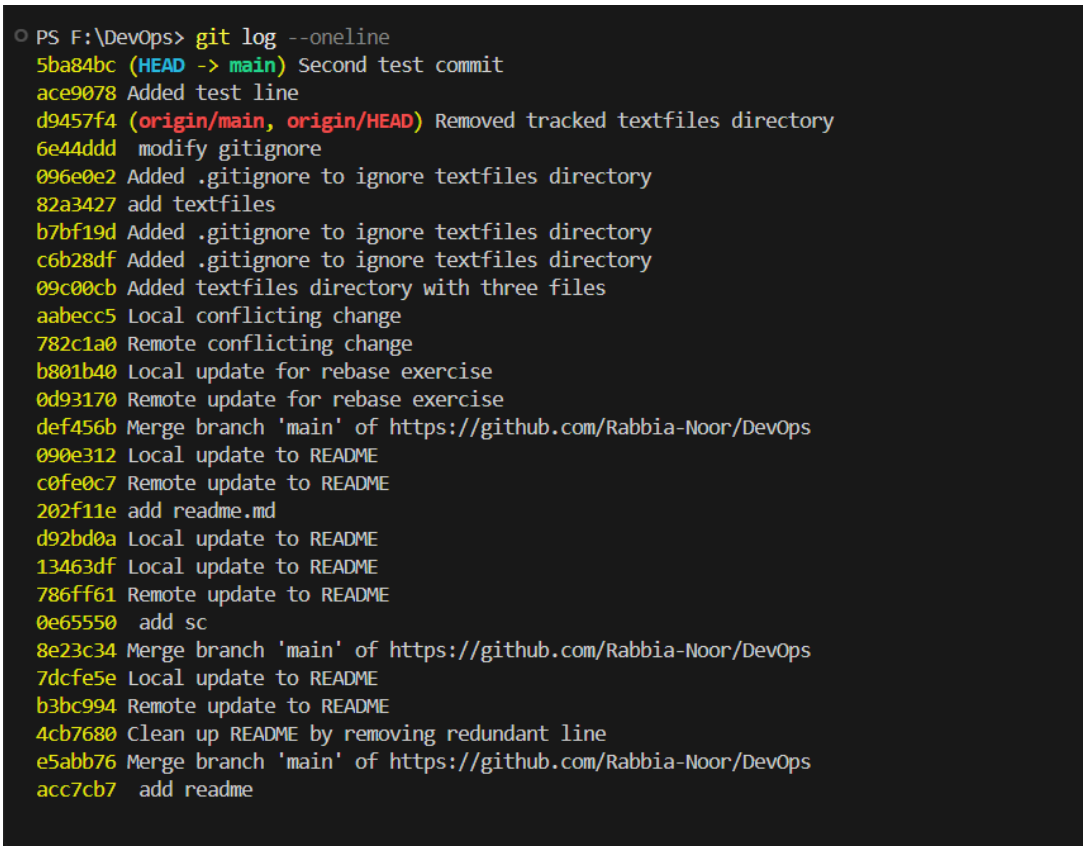
Step 1: Added a new line to a file and committed it as "Added test line".



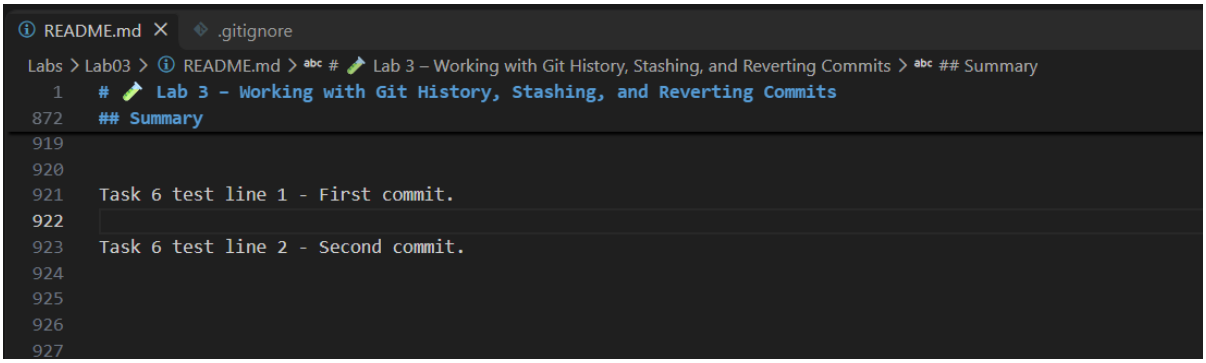
Step 2: Added another line and committed it as "Second test commit".



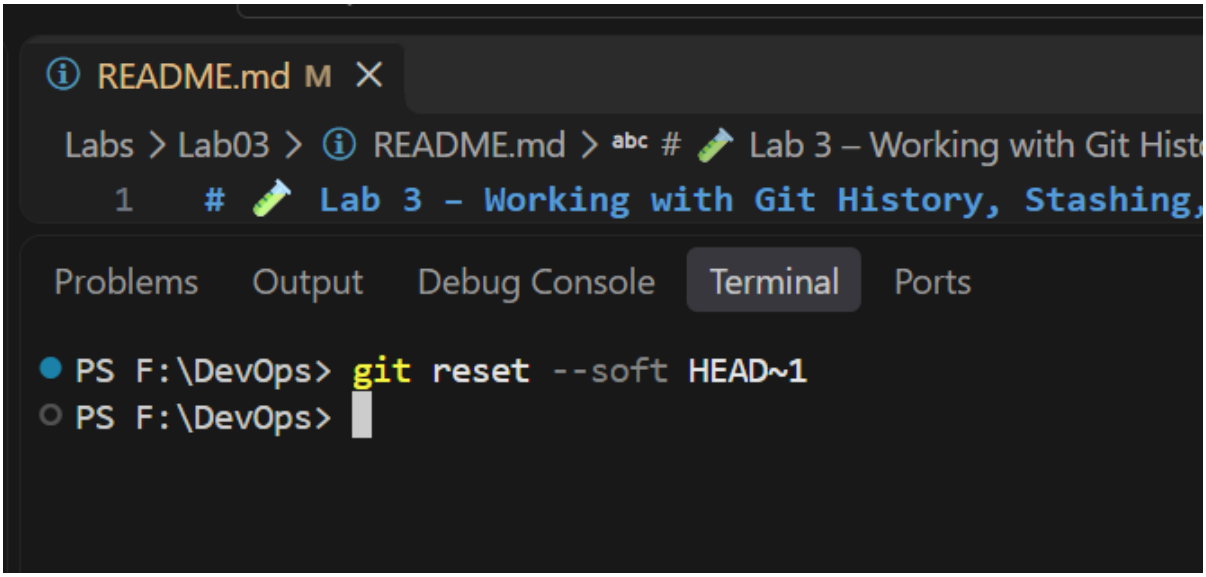
Step 3: Commit history before any reset, showing both test commits.



Step 4: The file contains both added lines before the reset.



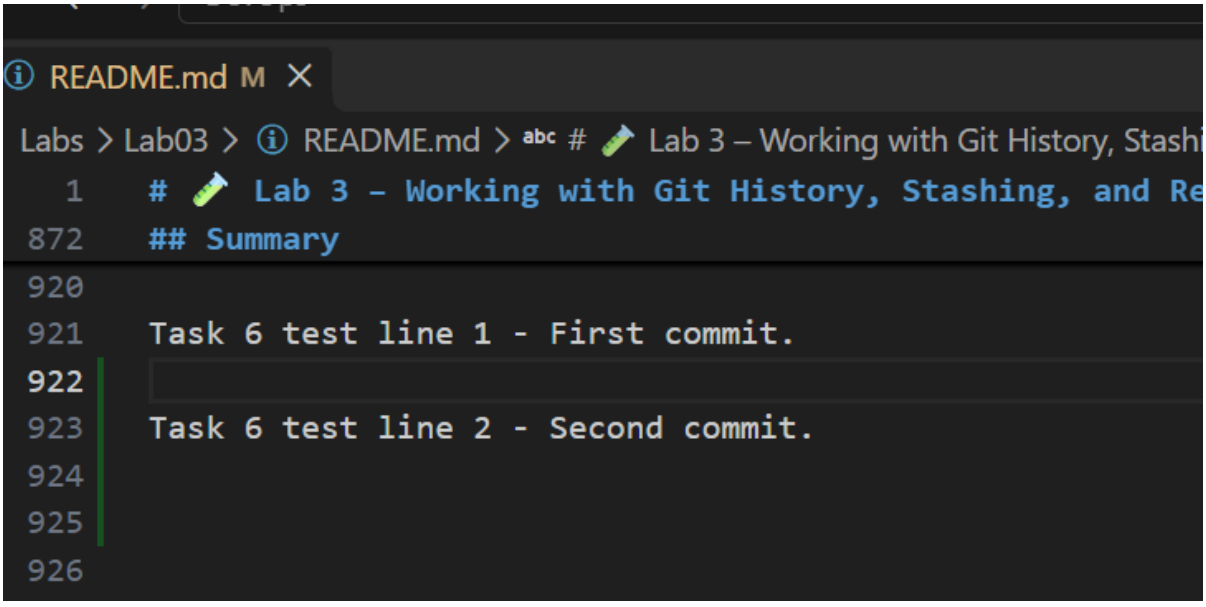
Step 5: Ran git reset --soft HEAD~1. The last commit is undone, but its changes stay staged.



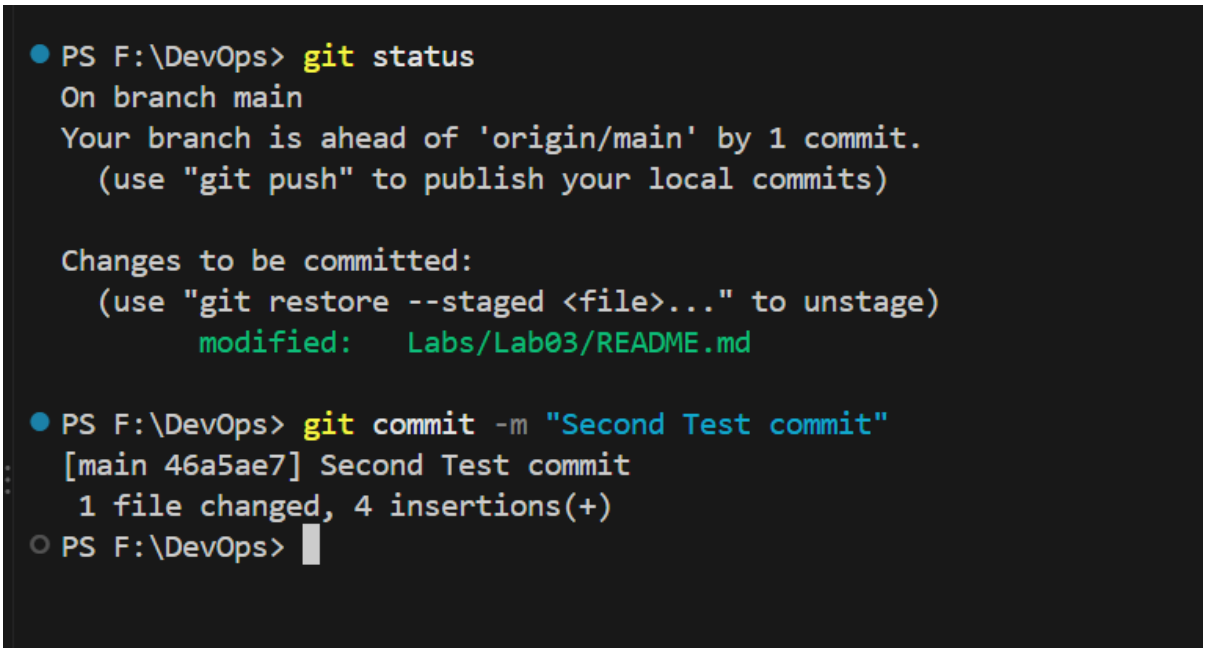
Step 6: History after the soft reset: the second commit no longer appears in the log.

```
PS F:\DevOps> git log --oneline
ace9078 (HEAD -> main) Added test line
d9457f4 (origin/main, origin/HEAD) Removed tracked textfiles directory
6e44ddd  modify gitignore
096e0e2 Added .gitignore to ignore textfiles directory
82a3427 add textfiles
b7bf19d Added .gitignore to ignore textfiles directory
c6b28df Added .gitignore to ignore textfiles directory
09c00cb Added textfiles directory with three files
aabecc5 Local conflicting change
782c1a0 Remote conflicting change
b801b40 Local update for rebase exercise
0d93170 Remote update for rebase exercise
def456b Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
090e312 Local update to README
c0fe0c7 Remote update to README
202f11e add readme.md
d92bd0a Local update to README
13463df Local update to README
786ff61 Remote update to README
0e65550  add sc
8e23c34 Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
7dcfe5e Local update to README
b3bc994 Remote update to README
4cb7680 Clean up README by removing redundant line
e5abb76 Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
```

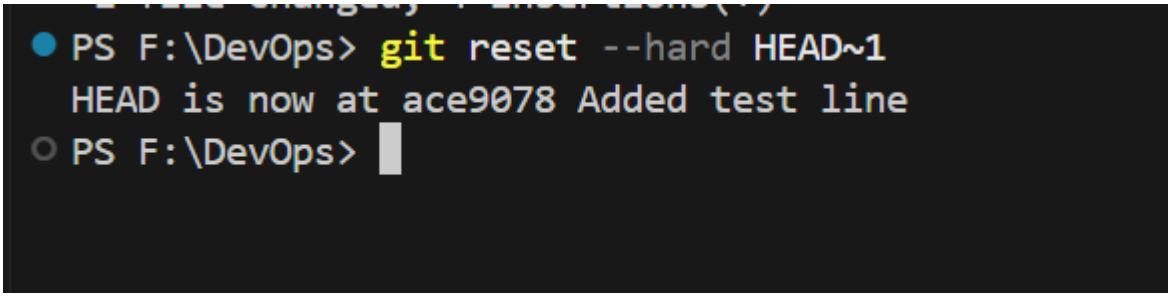
Step 7: The file still contains both edits, which shows the soft reset did not remove any changes.



Step 8: Committed the staged changes again as "Second Test commit" so a hard reset could be demonstrated.



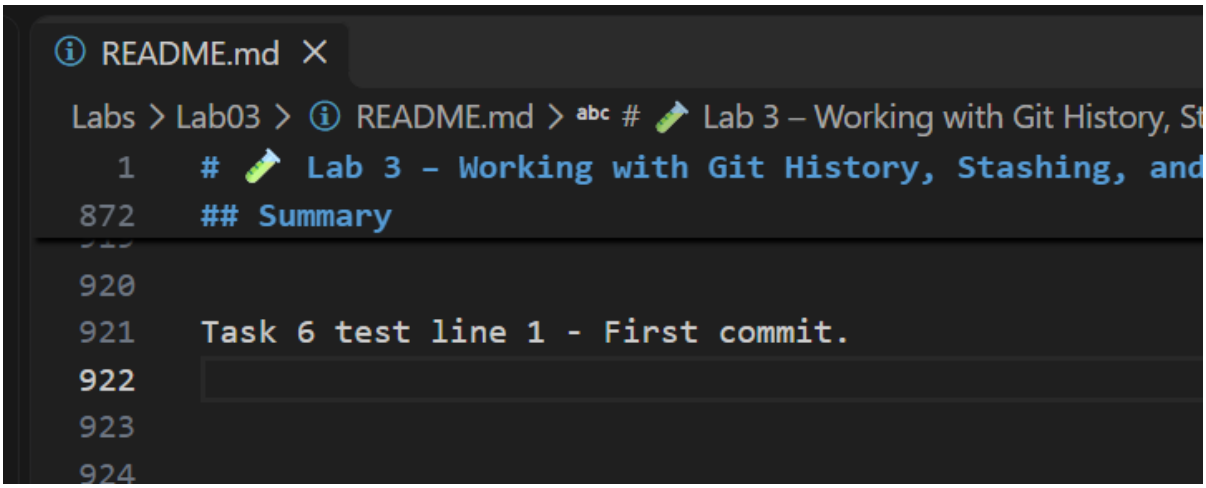
Step 9: Ran git reset --hard HEAD~1, which removes the last commit and its changes completely.



Step 10: History after the hard reset: the last commit is gone.

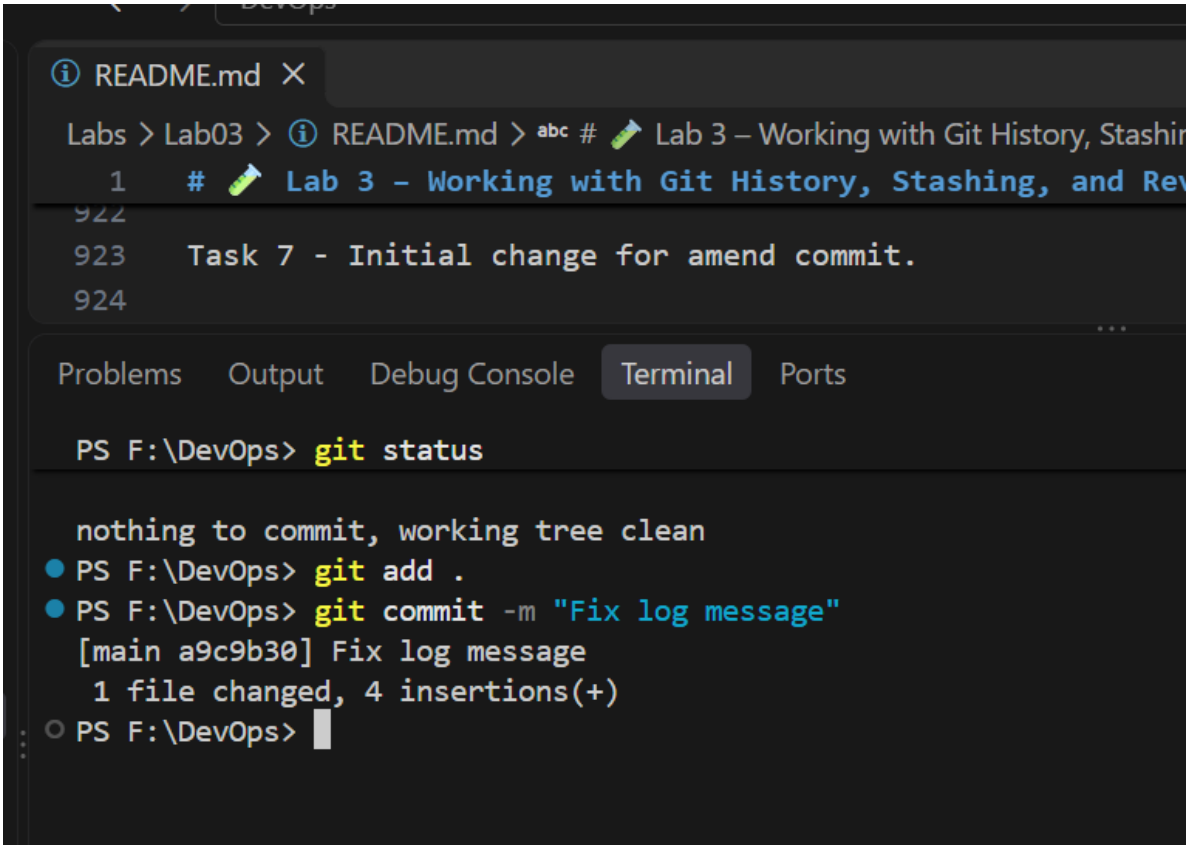
```
PS F:\DevOps> git log --oneline
ace9078 (HEAD -> main) Added test line
d9457f4 (origin/main, origin/HEAD) Removed tracked textfiles directory
6e44ddd  modify gitignore
096e0e2 Added .gitignore to ignore textfiles directory
82a3427 add textfiles
b7bf19d Added .gitignore to ignore textfiles directory
c6b28df Added .gitignore to ignore textfiles directory
09c00cb Added textfiles directory with three files
aabecc5 Local conflicting change
782c1a0 Remote conflicting change
b801b40 Local update for rebase exercise
0d93170 Remote update for rebase exercise
def456b Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
090e312 Local update to README
c0fe0c7 Remote update to README
202f11e add readme.md
d92bd0a Local update to README
13463df Local update to README
786ff61 Remote update to README
0e65550  add sc
8e23c34 Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
7dcfe5e Local update to README
b3bc994 Remote update to README
4cb7680 Clean up README by removing redundant line
e5abb76 Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
acc7cb7  add readme
```

Step 11: The file after the hard reset. The second edit has been removed and the file is back to the earlier commit.

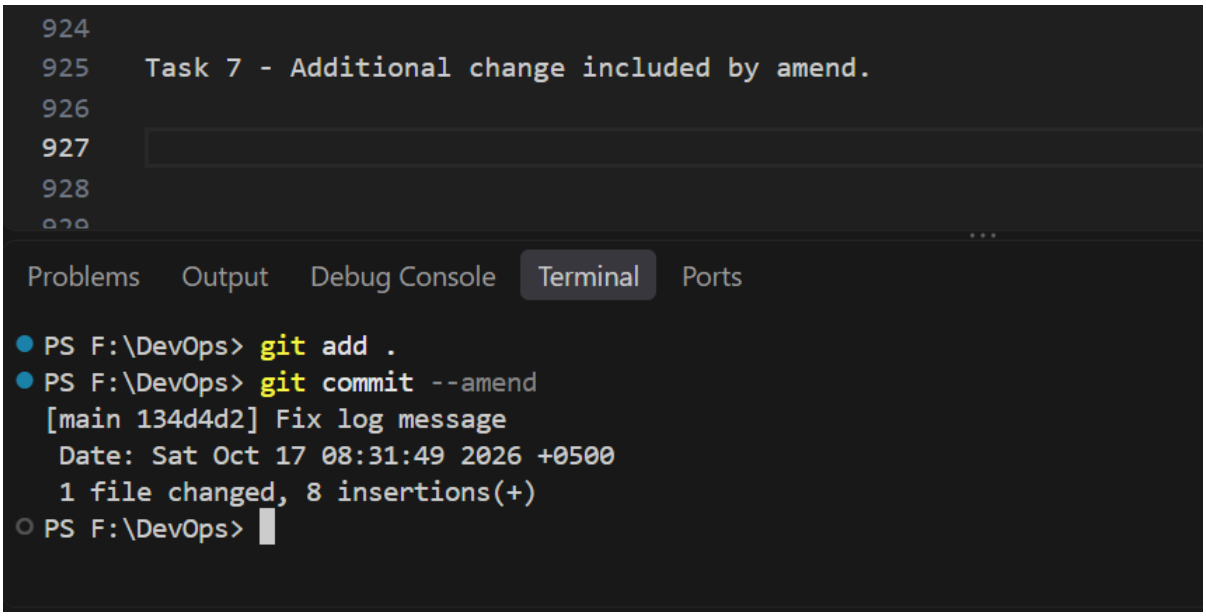


Task 7 – Amending the Last Commit

Step 1: Made a small change and committed it with the message "Fix log message".



Step 2: Updated another file that was forgotten, added it and ran git commit --amend. The change is merged into the previous commit.



Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

Step 1: Added "temporary text" to temp.txt, then committed and pushed it to the remote.

```
 1 file changed, 8 insertions(+)
PS F:\DevOps> echo "temporary text" >> temp.txt
PS F:\DevOps>  echo "temporary text" >> Labs/Lab03/temp.txt
PS F:\DevOps> git add .
PS F:\DevOps> git commit -m "Added temporary text file"
[main c2a21cb] Added temporary text file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Labs/Lab03/temp.txt
PS F:\DevOps> git push origin main
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 4 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (15/15), 1.25 KiB | 55.00 KiB/s, done.
Total 15 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), completed with 3 local objects.
To https://github.com/Rabbia-Noor/DevOps.git
   d9457f4..c2a21cb  main -> main
PS F:\DevOps>
```

Step 2: Ran git revert with the commit hash. Git created a new commit that undoes the change without deleting history.

```
PS F:\DevOps> git log --oneline
c2a21cb (HEAD -> main, origin/main, origin/HEAD) Added temporary text file
134d4d2 Fix log message
ace9078 Added test line
d9457f4 Removed tracked textfiles directory
6e44ddd  modify gitignore
096e0e2 Added .gitignore to ignore textfiles directory
82a3427 add textfiles
b7bf19d Added .gitignore to ignore textfiles directory
c6b28df Added .gitignore to ignore textfiles directory
09c00cb Added textfiles directory with three files
aabecc5 Local conflicting change
782c1a0 Remote conflicting change
b801b40 Local update for rebase exercise
0d93170 Remote update for rebase exercise
def456b Merge branch 'main' of https://github.com/Rabbia-Noor/DevOps
090e312 Local update to README
c0fe0c7 Remote update to README
202f11e add readme.md
d92bd0a Local update to README
13463df Local update to README
786ff61 Remote update to README
PS F:\DevOps> git revert c2a21cb
[main 7d7ce77] Revert "Added temporary text file"
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 Labs/Lab03/temp.txt
PS F:\DevOps>
```

Step 3: Pushed the revert commit to the remote with git push origin main.

```
PS F:\DevOps> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 369 bytes | 184.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Rabbia-Noor/DevOps.git
   c2a21cb..7d7ce77  main -> main
PS F:\DevOps>
```

Task 9 – Force Push (With Caution)

Step 1: Created a new branch for testing using git checkout -b test-force.

```
PS F:\DevOps> git checkout -b test-force
Switched to a new branch 'test-force'
PS F:\DevOps>
```

Step 2: Made a small change and committed it on the test-force branch.

```
PS F:\DevOps> echo "Temporary force-push test" > Labs/Lab03/force-test.txt
PS F:\DevOps> git add Labs/Lab03/force-test.txt
PS F:\DevOps> git commit -m "Add force push test file"
[test-force d75fdbd] Add force push test file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Labs/Lab03/force-test.txt
PS F:\DevOps>
```

Step 3: Pushed the test-force branch to the remote repository.

```
PS F:\DevOps> git push origin test-force
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 488 bytes | 244.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'test-force' on GitHub by visiting:
remote:      https://github.com/Rabbia-Noor/DevOps/pull/new/test-force
remote:
To https://github.com/Rabbia-Noor/DevOps.git
 * [new branch]      test-force -> test-force
PS F:\DevOps>
```

Step 4: Ran git reset --hard HEAD~1 to remove the last commit locally, so the local branch is now behind the remote.

```
PS F:\DevOps> git reset --hard HEAD~1
HEAD is now at 7d7ce77 Revert "Added temporary text file"
PS F:\DevOps>
```

Step 5: A normal git push was rejected by the remote because the local and remote histories no longer match.

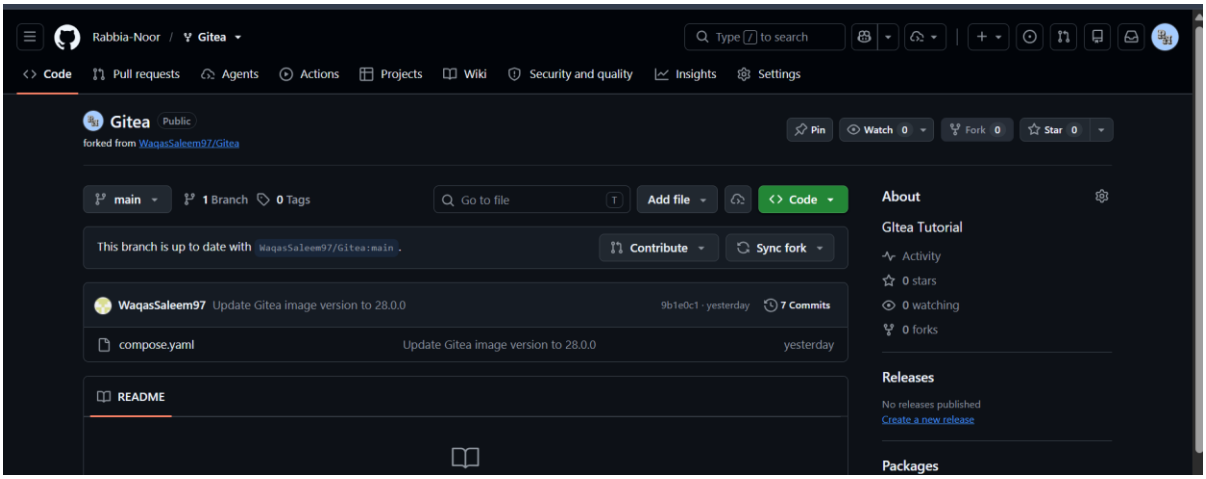
```
PS F:\DevOps> git push origin test-force
To https://github.com/Rabbia-Noor/DevOps.git
 ! [rejected]        test-force -> test-force (non-fast-forward)
error: failed to push some refs to 'https://github.com/Rabbia-Noor/DevOps.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS F:\DevOps>
```

Step 6: Ran git push origin test-force --force. The remote branch history was overwritten with the local one.

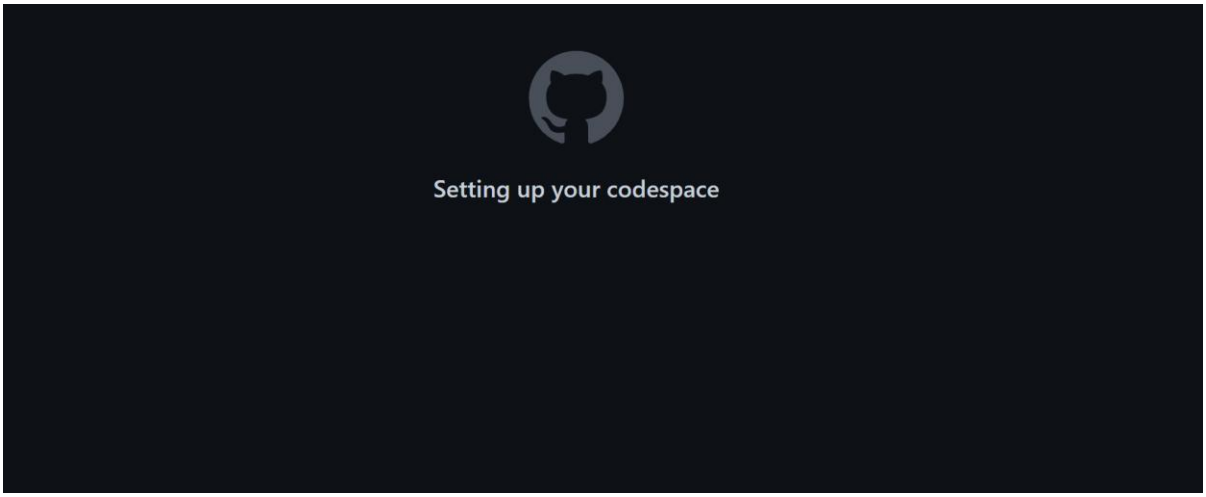
```
PS F:\DevOps> git push origin test-force --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Rabbia-Noor/DevOps.git
 + d75fdbd...7d7ce77 test-force -> test-force (forced update)
PS F:\DevOps>
```

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

Step 1: Forked the Gitea repository into my own GitHub account.



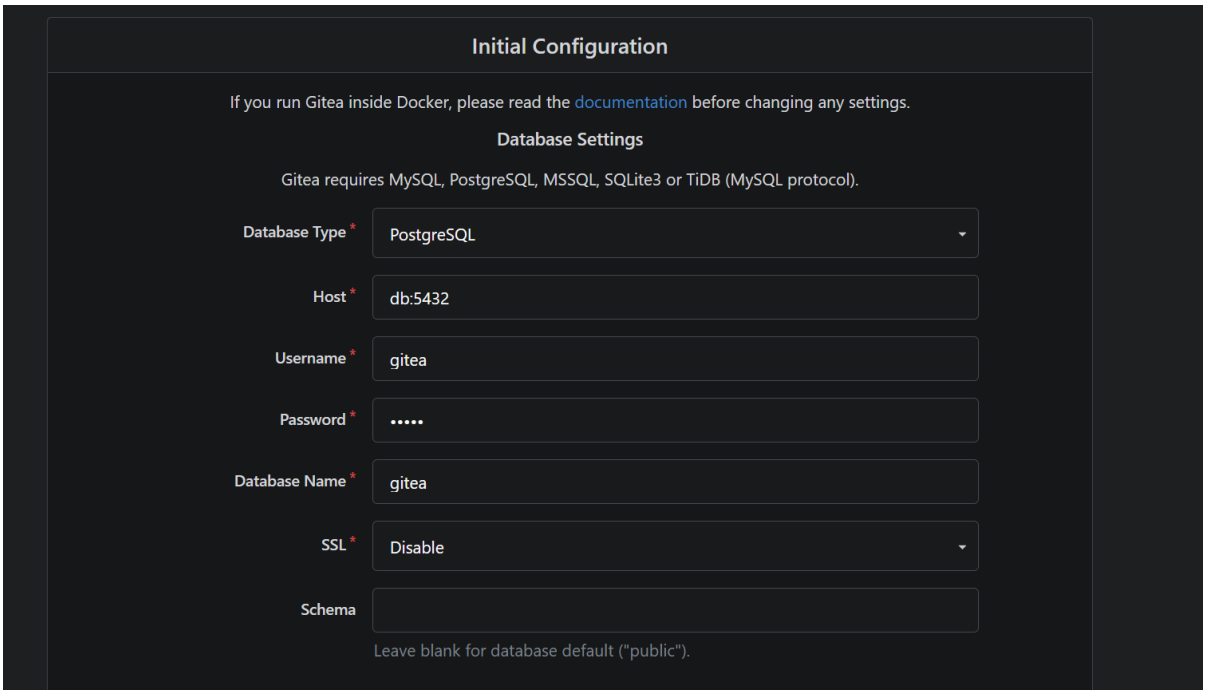
Step 2: Created a GitHub Codespace from the forked repository. The screenshot shows it loading.



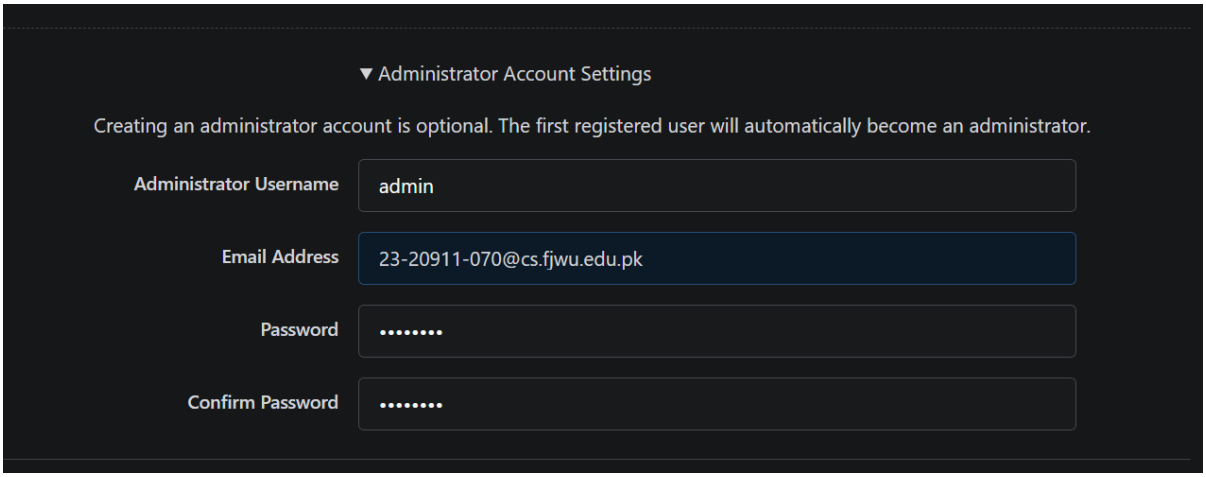
Step 3: Switched to the codespace branch and ran docker compose up -d. The Gitea containers started and are running.



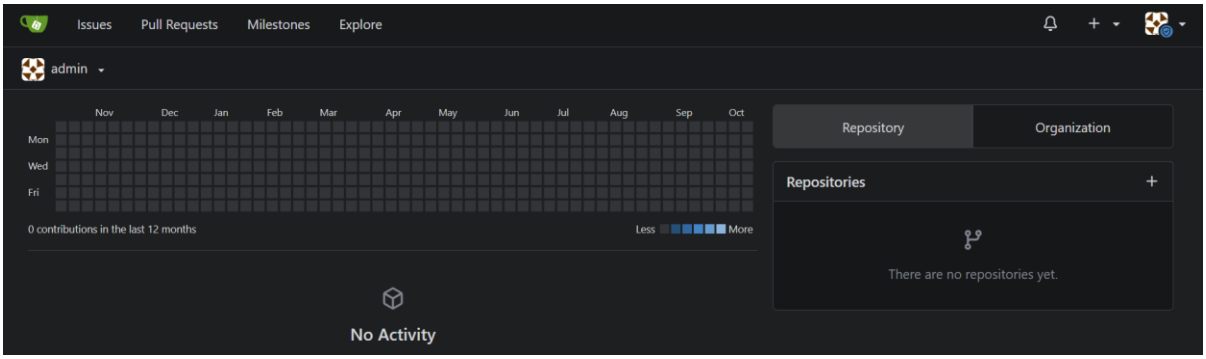
Step 4: Opened the forwarded port 3000 in the browser. The Gitea Initial Configuration page shows the database settings.



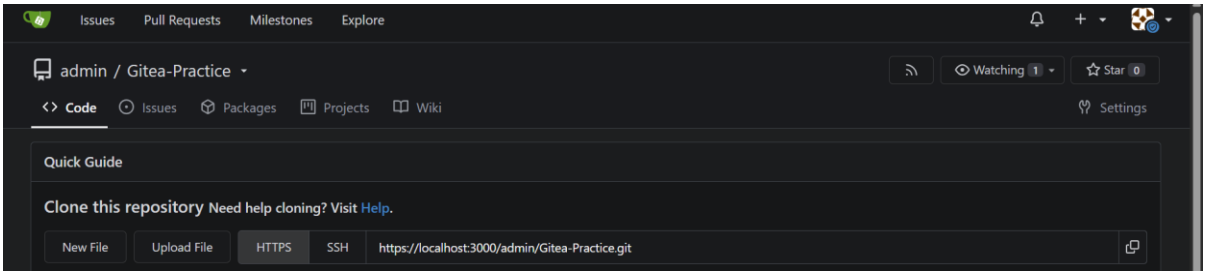
Step 5: Filled in the Administrator Account Settings with the admin username, email and password.



Step 6: Logged in with the admin account. The screenshot shows the Gitea dashboard.

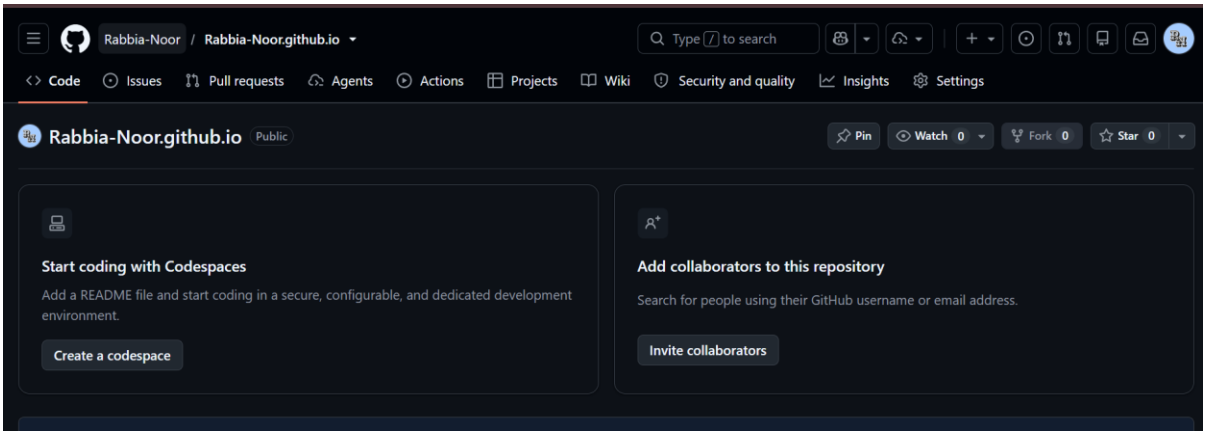


Step 7: Created a new repository inside Gitea.

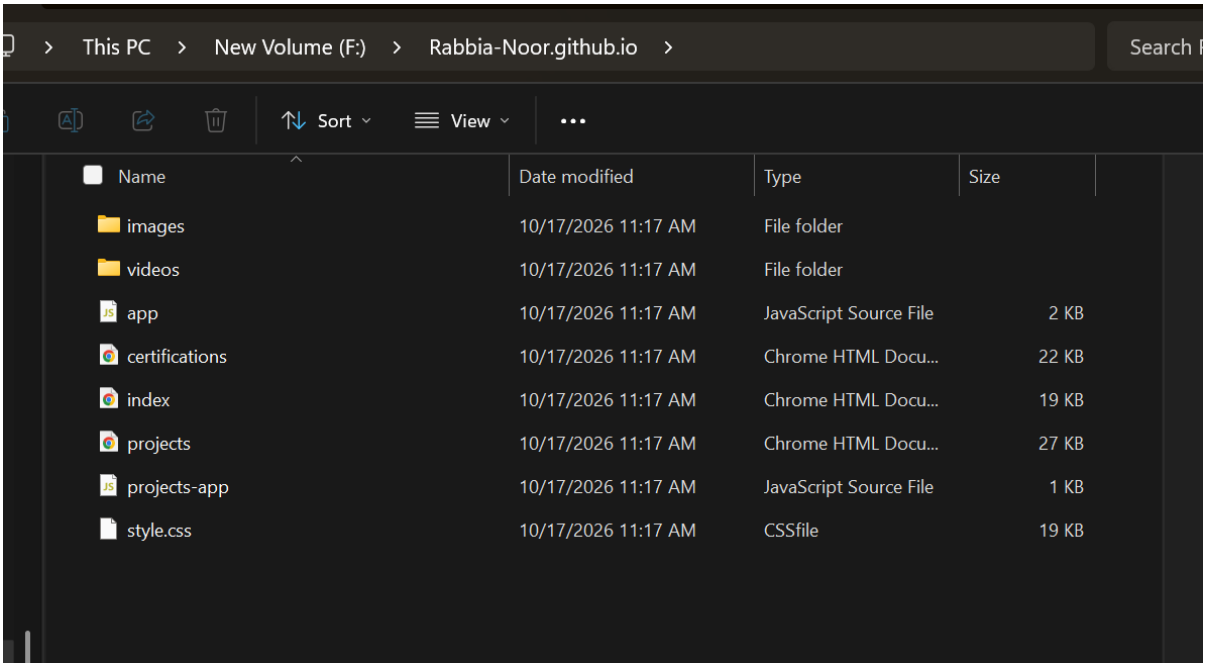


Task 11 – Creating a GitHub Pages Portfolio Site

Step 1: Created a public repository named <username>.github.io on GitHub.



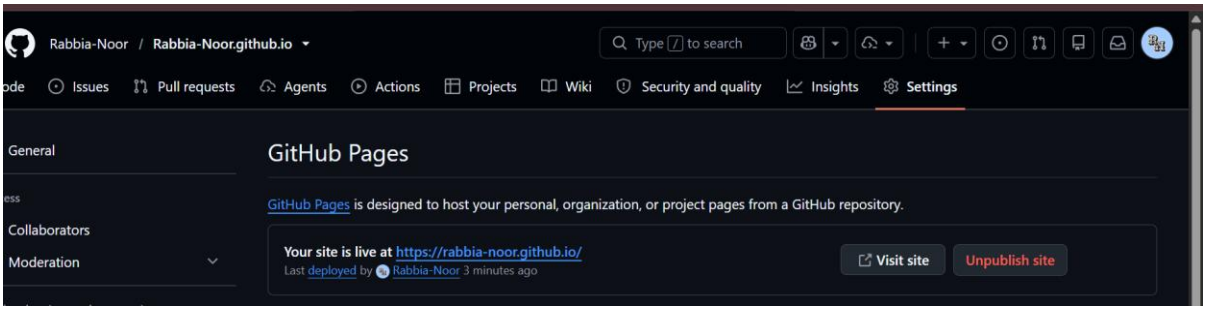
Step 2: The static website files (HTML, CSS and JS) prepared in a local folder.



Step 3: Initialised git, committed the site files and pushed them to the GitHub repository.

```
origin  https://github.com/Rabbia-Noor/Rabbia-Noor.github.io.git (push)
PS F:\Rabbia-Noor.github.io> git push -u origin main
Enumerating objects: 41, done.
Counting objects: 100% (41/41), done.
Delta compression using up to 4 threads
Compressing objects: 100% (37/37), done.
Writing objects: 100% (41/41), 40.61 MiB | 151.00 KiB/s, done.
Total 41 (delta 4), reused 41 (delta 4), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/Rabbia-Noor/Rabbia-Noor.github.io.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS F:\Rabbia-Noor.github.io>
PS F:\Rabbia-Noor.github.io>
```

Step 4: Settings > Pages shows that the site is published and gives the live link.



Step 5: The live portfolio site opened in the browser through the GitHub Pages link.

